SLM Math Storyteller — Complete UI Guide, Live Demo Script, and Viva Preparation

Project: SLM Math Storyteller with Retrieval-Augmented Generation **Team:** Fahad Ali Aslam Awan (537146), Muhammad Afnan (537042), Salman Ahmad (537135) **Supervisor:** Dr. Wajahat Hussain **Department of Electrical Engineering, NUST — April 2026**

This document is the companion to the LaTeX report. It is written so that any team member can pick it up the night before the viva, walk through the running app, and answer any question the panel is likely to ask.

Part 1 — How to Run the App

1.1 First-Time Setup

```
# 1. Clone / unzip the project, then enter it
cd RAG_3

# 2. Create a virtual environment (recommended)
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# 4. Place the GGUF model in models/
#    File: qwen2.5-1.5b-instruct-q4_k_m.gguf
#    (The path is set in config.yaml.)

# 5. Build the FAISS index from the knowledge base
python create_index.py
# Expected output: "  Index saved → faiss_index/ (N vectors)"

# 6. Launch the app
python app.py
# Gradio prints a local URL (e.g. http://127.0.0.1:7860). Open it.
```

1.2 What Happens When You Hit `python app.py`

1. **Config load** — `config.yaml` is parsed.
2. **LLM load** — Qwen 2.5-1.5B is memory-mapped from the GGUF file via `llama.cpp`. (~30 seconds, one-time.)

3. **Embedding model load** — `all-MiniLM-L6-v2` is downloaded the first time, cached afterwards.
4. **FAISS index load** — `faiss_index/index.faiss` is loaded into memory.
5. **spaCy model load** — `en_core_web_sm` is loaded for entity extraction.
6. **Gradio server** — UI mounts at `http://127.0.0.1:7860` and opens in the default browser.

If the status footer at the bottom of the page shows `System ready`, everything is up.

Part 2 — The UI, Tab by Tab, Feature by Feature

The app is a **single-page Gradio Blocks layout** divided into a left **sidebar** and a right **main panel**. There are no separate tabs; all controls are visible at once. Below, each control is described with **what it does, how to use it, and what to look for**.

2.1 Header

> **SLM Math Storyteller — RAG Enhanced** *Co-creative math adventures powered by a small language model with retrieval-augmented generation*

The header confirms branding. No interaction.

2.2 Sidebar — Settings Block

(a) `Knowledge Base (RAG)` checkbox

- **Default:** ON (checked).
- **Purpose:** Toggles retrieval. When ON, before each generation the user message is embedded and the top-3 chunks from `Math_Project/` are pulled and injected as context. When OFF, the model answers from its pretraining alone.
- **How to demo:** Type `What is the hero's lucky number?` once with RAG ON and once with RAG OFF.
  - **ON:** answer is **10** (cited from `Hero_Rules.txt`).
  - **OFF:** the model says it does not have that information, or invents a number.
- **Why this matters:** This is the cleanest possible demonstration of grounding.

(b) `Grade Level` dropdown (1 to 6)

- **Default:** 3.

- **Purpose:** Controls the difficulty of the embedded math problem. The system prompt literally contains *"Include ONE simple math problem fitting grade {grade}"*.
- **How to demo:** Select Grade 1 → ask for a dragon story → the math is `4 + 3 = 7`. Select Grade 6 → ask the same prompt → the math becomes a percentage / ratio problem.

2.3 Sidebar — Quick Start Block

Six buttons, each pre-filled with a story-starter prompt:

Button	Prompt sent
Dragon Quest	Tell me a story about a dragon who counts treasure
Pirate Fractions	A pirate finds a map with fraction puzzles
Space Math	An astronaut learns multiplication in space
Fairy Geometry	A fairy builds a castle using geometry
Robot Ratios	A robot discovers ratios while cooking
Math Riddle	Give me a fun math riddle to solve

Click → the prompt is sent immediately (you don't see it appear in the textbox; it goes straight to the chat). Use these to start a session quickly during the demo.

2.4 Sidebar — Live Metrics Block

Re-rendered after every chat turn. Six lines:

Metric	Meaning	What "good" looks like
Turns	Number of completed user/AI exchanges in this session	Climbs by 1 each turn
Math Error Rate	(incorrect facts) / (total facts) across the whole session	Should stay low; ideally 0 because of the verifier
Avg Topic Drift	Mean of `1 - cos(prev_emb, curr_emb)` across all turn-pairs	Below 0.4 once the session settles
Drift Trend	Linear-regression slope across all per-turn drifts (only after 4+ turns)	Negative = settling, positive = wandering

Metric	Meaning	What "good" looks like
Entity Consistency	Fraction of previously-introduced characters/places that recur in a sliding window	Close to 1.0 means the model is keeping the cast straight
Drift Alerts	Count of turns where per-turn drift exceeded the 0.4 threshold	Each alert is a topic shift; usually intentional (e.g. you asked a brand-new question)

These six numbers are the **dashboard for the viva**. When the panel asks "how do you know the system actually works?", you point here.

2.5 Sidebar — `New Session` button

- Clears chat history, entity sets, drift scores, and the math log.
- Does **not** reload the model (instant).
- Use it between demos so each one starts on a clean slate.

2.6 Sidebar — Rate This Session Block

Four 1-to-5 sliders followed by a Submit button:

- **Flow** — does the conversation flow naturally?
- **Consistency** — same characters/places used?
- **Engagement** — would a child enjoy this?
- **Math Accuracy** — are the math problems solved correctly?

Click **Submit Score** → the rating is appended to `evaluation_results/human_scores.json`. This is the human-evaluation channel that complements the automatic metrics.

2.7 Main Panel — Chat Window

- Turn-based, user messages on the right, AI on the left with a avatar.
- Under each AI reply there is a small italic line, e.g.: */ 18.4s / Characters: Glimmer / Places: Castle*
  - The first token is **the verification badge**: if math passed, Corrected if SymPy rewrote a number.
  - The second token is the **latency** in seconds.
  - Optional `Characters:` and `Places:` lists show entities accumulated so far (max 3 each).

2.8 Main Panel — Input Box and Send Button

- Type any prompt and press **Enter** or click **Send** .

- Multi-line is supported but the box is configured to one row to keep the layout tight.

2.9 Main Panel — Status Bar (bottom)

Two coloured pills, side by side:

Status pill	Meaning
Ready	Idle, waiting for input
Math verified	Last reply had no math errors
Math corrected	SymPy rewrote at least one fact in the last reply
Error	Engine returned an error (rare — usually a missing model file)
New session	Just clicked New Session

RAG pill	Meaning
RAG: ON	Retrieval active
RAG: OFF	No retrieval

The two pills together tell the user, at a glance, whether the system is grounded and whether the math just got fixed.

Part 3 — Live Demo Script (5 minutes)

Use this as a **runbook** — every step has a purpose and an expected outcome. If something does not match the expected outcome, the recovery line is in italics.

Step 0 — Pre-demo checklist (before the panel arrives)

☐ App is running, browser is on the Gradio URL.
☐ Sidebar shows `Turns: 0`, status bar shows `Ready`, RAG: ON.
☐ Make the chat window full-screen if possible.

Step 1 — Show the canary (RAG grounding)

> **Say:** "Let me start by showing why retrieval matters. I'll ask the system about a fact that lives only in our knowledge base."

1. Make sure **RAG is ON** and grade is **3**.
2. Type: `What is the hero's lucky number?`
3. **Expected:** The reply contains **10** and (often) the source file name.

4. Uncheck **RAG**.
5. Click **New Session**.
6. Ask the same question again.
7. **Expected:** The model says it does not have that information, or makes up a different number.

   Say: "That single fact lives in `Hero_Rules.txt`. With RAG, the model finds it. Without RAG, it cannot. This is the cleanest possible demonstration of grounding."

If the RAG-on reply does not say 10: turn RAG off and on once, click New Session, and re-ask. The retrieval index is sometimes warming up.

Step 2 — Show grade-aware story telling

1. Re-enable **RAG**, click **New Session**.
2. Set grade to **1**.
3. Click the **Dragon Quest** quick-start.
4. **Expected:** Short story (2-4 sentences), with simple math like `2 + 3 = 5`, ending with a continuation prompt.
5. Click **New Session**, set grade to **6**, click Dragon Quest again.
6. **Expected:** A more complex math problem — percentages, ratios, or a multi-step word problem.

   Say: "The grade selector flows into the system prompt, so the same story-starter produces grade-appropriate math."

Step 3 — Show math verification (the most impressive piece)

This one is best demonstrated by feeding the model a clearly-arithmetic question.

1. Ask: `if I have 8000 gems and someone steals 340, how many are left?`
2. **Expected:** The reply states `8000 - 340 = 7660` and the badge is .
3. Now ask: `now multiply by 230`
4. **Expected (real example from our log):** the model can produce a malformed product like "7660 x 230 = 1761800,1800". SymPy detects this; the displayed text is silently corrected; the badge becomes Corrected.

   Say: "Even when the small model stumbles on a big multiplication, the SymPy layer catches it. The user only ever sees the right number."

If the model happens to compute the multiplication correctly (it usually does for round numbers): point to the math-error metric in the sidebar and explain the fallback exists. Then ask a more challenging one such as `now divide that by 13` to provoke a non-trivial decimal.

Step 4 — Show coherence and entity tracking over a longer session

Continue the dragon thread for **at least four more turns** so the drift trend has data:

- `What was the gem count initially?` (model should say 8000)
- `Add 12 gems` (math fact)
- `Now divide by 100` (math fact)
- `If a thief stole all the gems, how many are left?` (model should say 0)

Then point to the sidebar:

> **Say:** "Notice three things. First, **Drift Trend is negative**, meaning the conversation is settling into a single storyworld instead of wandering. Second, **Entity Consistency is 1.0** — the dragon Glimmer that was introduced on turn 2 is still in the running set on turn 7. Third, the math metrics in the centre log every fact and every correction, so we have a per-session audit trail."

Step 5 — Show the human-evaluation panel

1. Drag the four sliders.
2. Click **Submit Score**.
3. Open `evaluation_results/human_scores.json` in a terminal — the score is there, with the turn count and grade attached.

> **Say:** "Automatic metrics are necessary but not sufficient. We also collect a four-axis human rating per session that we use to triangulate."

Step 6 — Show the offline evaluator

In a second terminal:

```
python evaluation.py
```

This reads `With_RAG.json` and `Without_RAG.json`, recomputes math accuracy, aggregates drift, and writes `evaluation_results/evaluation_report.md`.

> **Say:** "Every chat turn is logged to a JSONL file. The evaluator is fully offline and reproducible — anyone can re-derive the comparison table from the logs alone."

Step 7 — Wrap up

> **Say:** "To summarise: a 1.5 billion parameter model, on CPU, with retrieval, deterministic math verification, entity tracking, and drift monitoring, is reliable enough to be a co-creative math storyteller for grades 1 to 6. Everything runs locally. Nothing leaves the machine."

Part 4 — How Each Feature Maps to Each Project Objective

This section is the literal answer to "does this project meet its objectives?".

Objective (from Section 1.3 of the report)	Concrete feature that fulfils it	Where to point in the demo
Deploy a quantised SLM locally	Qwen 2.5-1.5B-Instruct Q4_K_M GGUF loaded by `llama.cpp`	Footer: `System: System ready`
Build a FAISS index over grade 1-6 KB	`create_index.py` + `Math_Project/` with 11 `.txt` files	Console output of `create_index.py` shows vector count
Implement deterministic math verification	`verify_math()` in `engine.py` using SymPy	The Corrected badge + the Math Error Rate metric
Track named entities (characters, places)	`EntityTracker` class (spaCy `en_core_web_sm`)	The "Characters: …" line under each AI reply
Quantify narrative coherence	`CoherenceTracker` with cosine drift + drift trend	Avg Topic Drift, Drift Trend, Drift Alerts in the sidebar
Provide a clean interactive UI	Gradio Blocks layout with sidebar + main panel	The whole UI
Log every turn for offline A/B analysis	JSONL writes to `With_RAG.json` / `Without_RAG.json`	`python evaluation.py`
Give the user a one-click RAG vs no-RAG comparison	The `Knowledge Base (RAG)` toggle	Demo Step 1 (the canary)

Every objective is wired to something the panel can see on screen. There is no "promised but not delivered" item.

Part 5 — Viva Q&A: Likely Questions and Crisp Answers

The questions below are grouped by theme. Each answer is short — long enough to be correct, short enough to recite under pressure.

5.1 Why this design?

Q: Why a 1.5B model and not a bigger one? A: Privacy and latency. A 1.5B Q4_K_M model fits in under 1.5 GB of RAM, runs on a laptop without a GPU, and never sends a child's conversation to a third-party server. The reliability layers (RAG + SymPy + entity + drift) are precisely what closes the quality gap with bigger models for this narrow domain.

Q: Why RAG and not fine-tuning? A: Three reasons. (1) Fine-tuning a 1.5B model on a couple of MB of curated math text would over-fit. (2) RAG lets us update the knowledge base without retraining. (3) RAG gives us source attribution at run-time — the model can cite `Grade3_Math.txt`. Fine-tuning cannot.

Q: Why FAISS and not Chroma / Pinecone / pgvector? A: FAISS is offline, embedded, zero-dependency, and fast for the index size we have (low hundreds of vectors). Chroma is also fine; we picked FAISS because LangChain's `FAISS.from_documents` is one line and the index serialises to two small files we can ship with the project.

Q: Why MiniLM-L6-v2 and not a larger embedding model? A: 384-dim embeddings are big enough for this content, the model is 22 MB, and we already needed a sentence encoder for the coherence tracker — using the same one keeps the dependency surface minimal.

Q: Why SymPy specifically? A: It is deterministic, audit-able, and handles rationals exactly. We do not need a neural verifier here — the arithmetic is grade-school, and a CAS is exactly the right tool. If SymPy says 7, it is 7.

Q: Why spaCy and not a transformer NER? A: Latency. `en_core_web_sm` is 12 MB and runs in milliseconds; a transformer NER would dominate the per-turn time. The recall is good enough for proper-noun characters and places, which is all we need.

5.2 How does RAG work in your system?

Q: Walk us through one chat turn end-to-end. A: (1) The user message is embedded with MiniLM. (2) FAISS returns the top-3 most similar chunks from `Math_Project/`. (3) Each chunk is prefixed with its source filename. (4) A grade-aware system prompt + the retrieved context + the last 6 chat exchanges + the new user message are passed to the SLM. (5) The model generates a draft. (6) SymPy scans the draft for `a op b = c` patterns and recomputes them; wrong results are rewritten in place. (7) spaCy extracts new entities; the running set is updated. (8) The coherence tracker stores the new turn's embedding and computes drift. (9) The corrected reply, the badges, and the metrics are returned to the UI; the turn is logged to JSONL.

Q: How do you choose top-k = 3? A: Empirically. With k=1, the model sometimes ignores the context. With k=5 the context window fills with irrele-

vant chunks. k=3 is the sweet spot for a 2048-token context.

Q: What is your chunking strategy? A: A 500-character splitter with 50-character overlap, applied per source file. The files themselves are short paragraphs separated by blank lines, so the chunk boundaries already align with conceptual units.

Q: Why include the source filename in the context? A: It costs almost no tokens and it lets the model cite the source — that is what produces replies like "According to `Grade3_Math.txt`, $4 \times 5 = 20$".

5.3 How does math verification work?

Q: Describe the verifier. A: Five regexes match `a + b = c`, `a - b = c`, `a × b = c`, `a ÷ b = c`, and `p/q = r/s`. Each match is recomputed with SymPy and compared to the model's stated answer with a 1e-6 tolerance. If wrong, the substring is replaced in the displayed text and the metrics dict logs `incorrect += 1`.

Q: What about word problems with no explicit equation? A: That is a known limitation. We catch every fact stated in the canonical `a op b = c` form. A multi-step problem solved purely in prose is not auto-corrected — the model has to state at least one numeric equation for us to verify it. Future work would add a step-by-step solver that re-derives the answer.

Q: What if the model writes math in LaTeX? A: The system prompt forbids LaTeX explicitly. We checked the logs — it doesn't.

Q: What if the model writes the same wrong fact twice in one reply? A: `text.replace(old, new, 1)` only replaces the first occurrence per fact. If the same wrong fact appears twice with the same operands, both are caught because the regex finds both matches and the loop runs once per match. We tested this.

5.4 How does coherence tracking work?

Q: What is "topic drift" exactly? A: For consecutive turns t-1 and t, drift = $1 - \cos(\text{embedding}(t-1), \text{embedding}(t))$. It is bounded in [0, 2] but for sentence-transformer embeddings it almost always lives in [0, 1]. A drift of 0 means identical topic; 0.4 is our empirical threshold for "noticeable shift".

Q: Why threshold 0.4? A: Calibrated against our own logs. Below 0.4, consecutive turns are clearly the same conversation. Above 0.4, a human reader would also call it a shift.

Q: What does drift trend tell you that average drift doesn't? A: Average drift can be moderate while the conversation is still wandering further every turn — a positive slope. Or it can be moderate because a few early turns drifted but the later ones settled — a negative slope. The trend distinguishes "still drifting" from "settled".

Q: How does entity consistency work? A: For each turn t, we look at the characters and places introduced before t (the "known" set) and the ones in a sliding window of 5 turns ending at t (the "recent" set). Consistency = |recent ∩ known| / |known|. Averaging over t gives one number. Closer to 1 means the cast is being maintained.

5.5 Evaluation

Q: How do you know your system is better than no RAG? A: Two ways. (1) The canary fact: `Hero_Rules.txt` says the hero's lucky number is 10. With RAG the answer is 10; without RAG, it isn't. (2) The A/B simulation in `evaluation.py --run` runs the same five seed prompts × 15 turns under both modes and reports math error rate, average drift, and drift trend side by side. We see lower math error rate and lower drift with RAG.

Q: What about human evaluation? A: The sidebar Rate-This-Session panel collects four 1-to-5 ratings per session. They are appended to `evaluation_results/human_scores.json` with the session metadata.

Q: What is the latency? A: ~15-25 s per reply on CPU-only Q4_K_M. RAG adds well under 100 ms. Latency is dominated by the LLM forward pass.

Q: Could you run faster? A: Yes — set `n_gpu_layers: -1` in `config.yaml` to push the model onto GPU; latency drops to a few seconds. We left it on CPU by default so the project runs anywhere.

5.6 Implementation choices

Q: Why one big `engine.py` instead of many small files? A: The whole runtime is one stateful object — model + index + entity tracker + coherence tracker + chat history. Splitting them across files would be nicer aesthetically but would not change behaviour and would make the imports messier. For a 500-line engine, one file is fine.

Q: Why Gradio and not Streamlit / Flask? A: Gradio's Blocks API gave us the sidebar + chat layout in about 80 lines and it has a built-in `Chatbot` component with avatars and history. Streamlit re-runs the whole script on every interaction, which would reload the model.

Q: Why store chat history as messages list and not a custom format? A: Because that is exactly the format the LLM expects. Zero conversion.

Q: Why JSONL logging? A: JSONL is the cheapest possible append-only format. A new line is a new turn. The evaluator just reads line-by-line; if a line is malformed, it skips and continues.

5.7 Failure modes and recovery

Q: What if the model file is missing? A: `_load_llm` raises `FileNotFoundError` with a message telling the user to download a GGUF and

put it in `models/` or edit `config.yaml`. The UI shows Error in the footer.

Q: What if the FAISS index is missing? A: `_load_rag` falls back to `_build_index`, which rebuilds it from the knowledge base. The user sees a one-time delay; subsequent launches are fast.

Q: What if spaCy is not installed? A: The entity tracker degrades gracefully — it returns an empty `{characters: [], places: []}`. The rest of the system keeps working; only the entity badges and consistency metric stop updating.

Q: What if the model produces a reply with no math at all? A: `verify_math` returns the text unchanged with `total_facts: 0` and `error_rate: 0.0`. The verification badge is by default.

5.8 Theory questions

Q: What is the difference between an SLM and an LLM? A: Practically, SLMs are roughly under 7B parameters and can run on consumer hardware; LLMs are 70B+ and need server-class GPUs. The architectural ideas are the same — both are decoder-only transformers — the difference is scale, training data, and deployment footprint.

Q: What is quantisation? What does Q4_K_M mean? A: Quantisation reduces the precision of model weights. Q4 means 4-bit; K refers to the GGUF "k-quant" family with mixed-precision blocks; M is the medium variant. Q4_K_M shrinks the model ~4x with a small accuracy loss.

Q: How do sentence embeddings work? A: A pretrained transformer is fine-tuned with a contrastive objective so that semantically-similar sentences land close together in vector space. For MiniLM-L6-v2, "close together" means high cosine similarity in 384-dim space.

Q: How does FAISS make retrieval fast? A: FAISS organises vectors into structures (flat, IVF, HNSW) that let you find approximate nearest neighbours in sub-linear time. For our few-hundred-vector index, even flat search is essentially instantaneous.

Q: What is hallucination, and how do you mitigate it here? A: Hallucination is when the model emits content that is fluent but factually wrong. We mitigate it three ways: (1) RAG provides the source-of-truth for grade-specific facts, (2) SymPy verifies and silently corrects every arithmetic claim, (3) the system prompt tells the model to say "I do not have that info" when no relevant context is given. The combination handles factual hallucination (RAG), computational hallucination (SymPy), and refusal hallucination (prompt).

Q: What is RAG vs fine-tuning vs prompting? A: *Prompting* puts the knowledge in the prompt directly — works for tiny domains. *RAG* fetches the right prompt content per query from a vector store — works for large or changing domains. *Fine-tuning* bakes knowledge into the weights — works when

you need style/format more than facts. We use RAG because our knowledge base is curated, growable, and best treated as data, not weights.

Q: What is cosine similarity? A: $\cos(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|)$. It measures the angle between two vectors, ignoring magnitude. Range [-1, 1]; for sentence embeddings it is practically [0, 1].

5.9 Trick / pressure questions

Q: If I delete `faiss_index/`, what happens? A: Next launch, the engine notices the index is gone and rebuilds it from `Math_Project/` automatically. Subsequent launches are fast again.

Q: If I make the knowledge base file empty, will the system still work? A: With RAG ON, the retrieval will return nothing useful; the model will fall back to its pretraining and answer as if RAG were OFF. The math verifier and entity tracker still run.

Q: What if I ask the model something unrelated to math? A: The system prompt has a "When answering GENERAL questions" branch that tells the model to answer briefly and helpfully. RAG retrieves whatever it considers most similar (often the wrong thing); the prompt explicitly says "If reference material is NOT relevant, ignore it".

Q: Could you replace Qwen 2.5 with a different model? A: Yes. Change `model.path` in `config.yaml` to any GGUF file. Or set `backend: openai` and point `openai.base_url` at LM Studio or any OpenAI-compatible server. The engine code does not care.

Q: Is the system safe for children? A: Three layers. (1) The system prompt forbids LaTeX, profanity, complex words, and over-long replies. (2) The knowledge base is hand-curated grade-school math content. (3) The model is small, instruction-tuned, and runs locally — no external content is ever fetched at runtime. There is no internet call from any chat turn.

Q: What happens if two users use the app at once? A: Gradio queues requests by default, and our `StoryEngine` is a single-process singleton, so two simultaneous users share the same chat history. For a multi-user deployment we would shard the engine per session — the engine class is already structured for that (each instance owns its own state).

Q: How would you scale this to 10,000 students? A: Three changes. (1) Per-session engine instances behind a session-id router. (2) GPU offload of the LLM, ideally with vLLM or llama-server batching. (3) A managed vector store (e.g. Qdrant) and a teacher dashboard that aggregates the per-session JSONL logs into class-level analytics.

Q: What is the single thing you would do differently next time? A: Extend the SymPy verifier to handle multi-step word problems. The current

regex catches single-line equations only. A symbolic step-by-step solver would close that last reliability gap.

Part 6 — Quick Reference: Key Numbers and Files

Files

File	Purpose
`app.py`	Gradio UI
`engine.py`	LLM + RAG + SymPy + spaCy + coherence
`config.yaml`	All knobs
`create_index.py`	One-shot FAISS builder
`evaluation.py`	Offline analysis + A/B simulation
`requirements.txt`	Python deps
`Math_Project/*.txt`	11 knowledge-base files
`models/*.gguf`	The local SLM
`faiss_index/`	Persisted vector store
`With_RAG.json`, `Without_RAG.json`	JSONL session logs
`evaluation_results/`	Aggregated reports + human scores

Key configuration knobs

- `model.path` — which GGUF
- `model.n_gpu_layers` — 0 = CPU, -1 = all on GPU
- `model.temperature` — default 0.7
- `model.max_tokens` — default 200
- `rag.top_k` — default 3
- `rag.chunk_size` / `chunk_overlap` — default 500 / 50
- `coherence.drift_threshold` — default 0.4
- `app.default_grade` — default 3

One-line summary you can recite

> "A 1.5B-parameter Qwen running locally, grounded by FAISS retrieval over a curated grade-1-to-6 math knowledge base, with deterministic SymPy math verification, spaCy entity tracking, and embedding-based drift monitoring, served through a Gradio UI with live metrics — fully offline, fully reproducible from JSONL logs."

That sentence is enough for the first 30 seconds of any answer.

Part 7 — Things to Have on Screen Before the Viva Starts

1. A **terminal** running `python app.py` (do not minimise — the panel may want to see the console).
2. A **browser tab** on the Gradio URL, scrolled so the sidebar and main panel are both visible.
3. A **second terminal** ready to run `python evaluation.py`.
4. A **file manager** (or `ls`) showing `With_RAG.json`, `Without_RAG.json`, `evaluation_results/`.
5. The **report PDF** in a tab of its own, in case the panel asks for a diagram.

Good luck.